Document number: P0881R7
Project: Programming Language C++

Alexey Gorgurov <leha-bot@yandex.ru>, <no-vista@yandex.ru>
Antony Polukhin, Yandex.Taxi Ltd, <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2020-09-16

A Proposal to add stacktrace library

I. Motivation

In the current working draft [N4741] there is no way to get, store and decode the current call sequence. Such call sequences are useful for debugging and post mortem debugging. They are popular in other programming languages (like Java, C#, Python).

Pretty often assertions can't describe the whole picture of a bug and do not provide enough information to locate the problem. For example, you can see the following message on out-of-range access:

```
boost/array.hpp:123: T& boost::array<T, N>::operator[](boost::array<T, N>::size_type): Assertion '(i < N)&&("out of range")' failed.
Aborted (core dumped)
```

That's not enough information in the assert message to locate the problem without debugger.

This paper proposes classes that could simplify debugging and may change the assertion message into the following:

```
Expression 'i < N' is false in function 'T& boost::array<T, N>::operator[](boost::array<T, N>::size_type)': out of range.
Backtrace:
 0# boost::assertion_failed_msg(char const*, char const*, char const*, char const*, long) at ../example/assert_handler.cpp:39
 1# boost::array<int, 5ul>::operator[](unsigned long) at ../../../boost/array.hpp:124
 2# bar(int) at ../example/assert_handler.cpp:17
 3# foo(int) at ../example/assert_handler.cpp:25
 4# bar(int) at ../example/assert_handler.cpp:17
 5# foo(int) at ../example/assert_handler.cpp:25
 6# main at ../example/assert_handler.cpp:54
 7# 0x00007F991FD69F45 in /lib/x86_64-linux-gnu/libc.so.6
 8# 0x0000000000401139
```

II. Design Decisions

The design is based on the Boost.Stacktrace library, a popular library that does not depend on any non-standard library components.

Note about signal safety: this proposal does not attempt to provide a signal-safe solution for capturing and decoding stacktraces. Such functionality currently is not implementable on some of the popular platforms. However, the paper attempts to provide extensible solution, that may be made signal safe some day by providing a signal safe allocator and changing the `stacktrace` implementation details.

Note on performance: during Boost.Stacktrace development phase many users requested a fast way to store stacktrace, without decoding the function names. This functionality is preserved in the paper. All the `stack_frame` functions and constructors are lazy and won't decode the pointer information if there was no explicit request from class user.

Note on allocations: initial (pre-Boost) implementations of Boost.Stacktrace were not using allocator and all the frames were placed inside a fixed size internal storage. That was a mistake! Sometimes the most important information is located at the bottom of the stack. For example if you run Boost.Test, then the test name will be located low on the stack. With a fixed size storage the bottom of the stack could be lost along with the information.

Current design assumes that by default users wish to see the whole stack and OK with dynamic allocations, because do not construct `stacktrace` in performance critical places. For those users, who wish to use `stacktrace` on a hot path or in embedded environments `basic_stacktrace` allows to provide a custom allocator that allocates on the stack or in some other place, where users thinks it is appropriate.

Custom allocators support for functions like `to_string`, `source_file` and `description` is not provided. Those functions usually require platform specific allocations, system calls and a lot of CPU intensive work. Custom allocator does not provide benefits for those functions as the platform specific operations take an order of magnitude more time than the allocation.

Note on returning `std::string` and not having `noexcept` on `stack_frame::source_line()`: Unfortunately this is a necessarity on some platforms, where getting source line requires allocating or where source file name returned into a storage provided by user.

Note on expected implementation: We assume that Standard Library implementations would allow to disable/enable gathering stacktraces by a compiler switch that does not require recompiling the whole project. In other words, we expect to see a compiler option like `-fno-stacktrace` or `libstacktrace/lib_stacktrace_noop` libraries with the same ABI that would force the constructor of the `basic_stacktrace` to do nothing. This feature is implemented in Boost.Stacktrace and is highly requested in big projects.

Should `stacktrace` be a class or a function?

class	function

`struct promise_type {`
`  std::vector<stack_frame> frames;`
 	
`  void append(const stacktrace& s) {`
	`struct promise_type {`
	`  std::vector<stack_frame> frames;`
	
	`  void append(const std::vector<stack_frame>& s) {`

```
        frames.insert(frames.end(), s.begin(), s.end());
    }

    void print() {
        for (int i=0; auto& frame: frames) {
            std::cout << i++ << "  " << frame;
        }
    }
};
```

```
        frames.insert(frames.end(), s.begin(), s.end());
    }

    void print() {
        std::cout << frames;
    }
};
```

```
class stacktrace {
    small_vector<stack_frame> frames_;
    platform-specific-cache-of-internals;

public:
    operator bool() const noexcept;
    // almost all the vector interface
};
```

```
template<class Allocator = allocator<stack_frame>>
vector<stack_frame, Allocator>
  stacktrace(const Allocator& alloc = Allocator{}) noexcept;

template<class Allocator = allocator<stack_frame>>
vector<stack_frame, Allocator>
  stacktrace(size_type skip, size_type max_depth,
    const Allocator& alloc = Allocator{}) noexcept;
```

```
template<class Allocator>
string to_string(const basic_stacktrace<Allocator>& st);

template<class charT, class traits, class Allocator>
basic_ostream<charT, traits>& operator<<
    (basic_ostream<charT, traits>& os,
        const basic_stacktrace<Allocator>& st);
```

```
template<class Allocator>
string to_string(const vector<stack_frame, Allocator>& st);

template<class charT, class traits, class Allocator>
basic_ostream<charT, traits>& operator<<
    (basic_ostream<charT, traits>& os,
        const vector<stack_frame, Allocator>& st);
```

```
    stacktrace s;
    if (s) {
        std::cout << "backtrace: " << s;
    }
```

```
    auto s = stacktrace();
    if (!s.empty()) {
        std::cout << "backtrace: " << s;
    }
```

LEWG decided to leave it a separate type: "Prefer stacktrace as a type rather than `vector`." SF:0, F:3, N:5, A:0, SA:0

III. Significant changes to review

Kona questions to LEWG:

1. `stack_frame` is actually a pointer to some instruction. It's a static frame, not a dynamic one (the latter including the local variables and coroutines etc). **Should `stack_frame` be renamed into ~~`invocation_info`~~ `stacktrace::entry`?** Also note that the name `stack_frame` could be usable in the coroutines and fibers worlds.

   LEWG: Naming for stack_frame:
   2 stack_frame
   3 invocation_info
   9 stacktrace::entry
   3 stacktrace::frame
   5 stacktrace_entry
   1 stacktrace_frame
   2 frame_descriptor
   4 call_info
   2 call_descriptor
   2 frame_info.
   stacktrace::entry wins (Would be backtrace::entry if rename occurs).

   LEWG: Nest class inside stacktrace: SF 4 F 3 N 1 A 3 SA 1. Not concensus, status quo is against.

2. If we now have `invocation_info`, **should `stacktrace` be renamed into `backtrace`?** Note that there is a `::backtrace` function on GNU Linux.

   LEWG: stacktrace vs backtrace: SS 1 S 4 N 2 B 3 SB 3. Author decides.

3. Querying information about the frame is a heavy operation. To be consistent with the Filesystem approach **should the query functions become a free functions?**

   LEWG: Leave expensive ops as member functions. Unanimous consent.

4. There is a difference between "physical" and "logical" invocations. In other words `std::stacktrace` may have size `N`, while the implementation could decode `N` `stack_frames` into `N+X` records in the `to_string` function. **Is LEWG OK with shipping the current design that gives no way to retrieve the logical invocations size?**

   LEWG: We are OK with the fact that N physical entries can become N+X logical entries after decoding (because of expanding inline functions). Unanimous consent.

stack_frame	backtrace::entry

```
unordered_map<stack_frame, string> cache;
// ...
for (stack_frame f : stacktrace{}) {
    auto it = cache.find(f);
    if (it == cache.end()) {
        it = cache.emplace(f, f.description())->first;
    }
    cerr << it->second;
}
```

```
unordered_map<backtrace::entry, string> cache;
// ...
for (backtrace::entry f : backtrace{}) {
    auto it = cache.find(f);
    if (it == cache.end()) {
        it = cache.emplace(f, description(f))->first;
    }
    cerr << it->second;
}
```

```
void assertion_failed_msg(char const* expr) {
  std::stacktrace st(1, 1);
  stack_frame inf = st[0];
  std::cerr << "Expression '" << expr << "' is false in '"
      << inf.description() << " at "
      << inf.source_file() << ':' << inf.source_line()
      << ".\n";
  std::abort();
}
```

```
void assertion_failed_msg(char const* expr) {
  std::backtrace st(1, 1);
  backtrace::entry inf = st[0];
  std::cerr << "Expression '" << expr << "' is false in '"
      << description(inf) << " at "
      << source_file(inf) << ':' << source_line(inf)
      << ".\n";
  std::abort();
}
```

```
struct promise_type {
    std::vector<stack_frame> trace;

    void append(const stacktrace& s) {
        trace.insert(trace.end(), s.begin(), s.end());
    }
    // ...
};
```

```
struct promise_type {
    std::vector<backtrace::entry> trace;

    void append(const backtrace& s) {
        trace.insert(trace.end(), s.begin(), s.end());
    }
    // ...
};
```

If the `invocation_info` and `stack_frame` are both not acceptable, the following input could be used to preview the above table with other names (like `call_info`,`trace_info`, `stacktrace::record`...): backtrace::entry

LEWG:

- Added hash support.
- Removed `stack_frame` constructors from function pointers and `stack_frame::native_ptr_t` alias.
- LWG requested `rbegin` like functions for `basic_stacktrace`, so now `basic_stacktrace` satisfies the requirements of an allocator-aware container, of a sequence container and reversible container except that only operations defined for const-qualified sequence containers are supported and that the semantics of comparison functions and default constructor are different from those required for a container.

LEWG was OK with the above changes.

SG16:

- `stack_frame::source_file()` and encodings.

SG16 discussed a number of options including the possibility of `source_file()` returning `std::filesystem::path`. SG16 converged on the following recommendation: "Align behavior with source_location; remove wording regarding conversion; string contents are implementation defined. ". No objection to unanimous consent.

CWG question to LEWG:

- `stack_frame::address()` function should return instruction pointer, or stack address (that may have contained the instruction pointer), or both?

LEWG in favour of instruction pointer: "`stack_frame::address()` should return (something like) the instruction pointer (only)." SF:4, F:7, N:0, A:0, SA:0.

Points of special interest for CWG:

- *stacktrace* definition in [stacktrace.def] and member functions in [stack_frame.query]. Wording should not prevent any optimizations.

IV. Wording Intent

Key features that should be preserved by implementations:

- All the functions are lazy and do not query the stacktrace entry information if there was no explicit request.
- No fixed max size for trace - all the available invokers must be stored in a `stacktrace`.
- Implementations should allow to disable/enable gathering stacktraces by a link-time switch.
- Stacktracing does not prevent any of the optimizations.
- No info for a stacktrace entry is OK (it's not an error, do not throw!).
- `stacktrace_entry::description()` should return a demangled function signature if possible.
- `to_string(stacktrace)` should query information from debug symbols, symbol export tables and any other sources, returning demangled signatures if possible.
- Providing information about inlined functions that have no separate stacktrace entries is welcomed in `to_string(stacktrace)`.
- Detecting inlined functions and doing other heavy operations is not welcomed in basic_stacktrace constructors or `stacktrace_entry::current`.
- `stacktrace` should be usable in contract violation handler, coroutines, handler functions [handler.functions], parallel algorithms, etc.

V. Wording

[Note for editor: Add the header <stacktrace> into the Table 19 in [headers] - ***end note]***

20.? Stacktrace [stacktrace]

20.?.1 General [stacktrace.general]

Subclause [stacktrace] describes components that C++ programs may use to store the stacktrace of the current thread of execution and query information about the stored stacktrace at runtime.

20.?.2 Stacktrace definition [stacktrace.def]

The *invocation sequence* of the current evaluation x_0 in the current thread of execution is a sequence $(x_0, \ldots, x_n)$ of evaluations such that, for `i >= 0`, x_i is within the function invocation x_{i+1} (6.8.1 [intro.execution]).

A *stacktrace* is an approximate representation of an invocation sequence and consists of *stacktrace entries*. A stacktrace entry represents an evaluation in a stacktrace.

20.?.3 Header <stacktrace> synopsis [stacktrace.syn]

```
namespace std {
  // 20.?.4, class stacktrace_entry
  class stacktrace_entry;

  // 20.?.5, class basic_stacktrace
  template<class Allocator>
  class basic_stacktrace;

  // basic_stacktrace typedef names
  using stacktrace = basic_stacktrace<allocator<stacktrace_entry>>;

  // 20.?.6, non-member functions
  template<class Allocator>
  void swap(basic_stacktrace<Allocator>& a, basic_stacktrace<Allocator>& b)
       noexcept(noexcept(a.swap(b)));

  string to_string(const stacktrace_entry& f);

  template<class Allocator>
  string to_string(const basic_stacktrace<Allocator>& st);

  template<class charT, class traits>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const stacktrace_entry& f);

  template<class charT, class traits, class Allocator>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const basic_stacktrace<Allocator>& st);

  // 20.?.7, hash support
  template<class T> struct hash;
  template<> struct hash<stacktrace_entry>;
  template<class Allocator> struct hash<basic_stacktrace<Allocator>>;
}
```

20.?.4 Class stacktrace_entry [stacktrace.entry]

20.?.4.1 Overview [stacktrace.entry.overview]

```
namespace std {
  class stacktrace_entry {
  public:
    using native_handle_type = implementation-defined;

    // 20.?.4.2, constructors
    constexpr stacktrace_entry() noexcept;
    constexpr stacktrace_entry(const stacktrace_entry& other) noexcept;
    constexpr stacktrace_entry& operator=(const stacktrace_entry& other) noexcept;

    ~stacktrace_entry();

    // 20.?.4.3, observers
    constexpr native_handle_type native_handle() const noexcept;
    constexpr explicit operator bool() const noexcept;

    // 20.?.4.4, query
    string description() const;
    string source_file() const;
    uint_least32_t source_line() const;

    // 20.?.4.5, comparison
    friend constexpr bool operator==(const stacktrace_entry& x, const stacktrace_entry& y) noexcept;
```

```
    friend constexpr strong_ordering operator<=>(const stacktrace_entry& x, const stacktrace_entry& y) noexcept;
  };
}
```

An object of type `stacktrace_entry` is either empty, or represents a stacktrace entry and provides operations for querying information about it. `stacktrace_entry` models `regular` [concepts.object] and `three_way_comparable<strong_ordering>` [cmp.concept].

20.?.4.2 Constructor [stacktrace.entry.cons]

```
constexpr stacktrace_entry() noexcept;
```

Postconditions: `*this` is empty.

20.?.4.3 Observers [stacktrace.entry.obs]

```
constexpr native_handle_type native_handle() const noexcept;
```

The semantics of this function are implementation-defined.

Remarks: Successive invocations of the `native_handle()` function for an unchanged `stacktrace_entry` object return identical values.

```
constexpr explicit operator bool() const noexcept;
```

Returns: `false` if and only if `*this` is empty.

20.?.4.4 Query [stacktrace.entry.query]

[Note: All the `stacktrace_entry` query functions treat errors other than memory allocation errors as "no information available" and do not throw in that case. *- end note]*

```
string description() const;
```

Returns: A description of the evaluation represented by `*this`, or an empty string.

Throws: `bad_alloc` if memory for the internal data structures or the resulting string cannot be allocated.

```
string source_file() const;
```

Returns: The presumed or actual name of the source file [cpp.predefined] that lexically contains the expression or statement whose evaluation is represented by `*this`, or an empty string.

Throws: `bad_alloc` if memory for the internal data structures or the resulting string cannot be allocated.

```
uint_least32_t source_line() const;
```

Returns: `0`, or a 1-based line number that lexically relates to the evaluation represented by `*this`. If `source_file` returns the presumed name of the source file, returns the presumed line number; if `source_file` returns the actual name of the source file, returns the actual line number.

Throws: `bad_alloc` if memory for the internal data structures cannot be allocated.

20.?.4.5 Comparison [stacktrace.entry.cmp]

```
friend constexpr bool operator==(const stacktrace_entry& x, const stacktrace_entry& y) noexcept;
```

Returns: `true` if and only if `x` and `y` represent the same stacktrace entry or both `x` and `y` are empty.

20.?.5 Class template `basic_stacktrace` [stacktrace.basic]

20.?.5.1 Overview [stacktrace.basic.overview]

```
namespace std {
  template<class Allocator>
  class basic_stacktrace {
  public:
    using value_type = stacktrace_entry;
    using const_reference = const value_type&;
    using reference = value_type&;
    using const_iterator = implementation-defined;
    using iterator = const_iterator;
    using reverse_iterator = std::reverse_iterator<iterator>;
    using const_reverse_iterator = std::reverse_iterator<const_iterator>;
    using difference_type = implementation-defined;
    using size_type = implementation-defined;
    using allocator_type = Allocator;

    // 20.?.5.2, creation and assignment
    static basic_stacktrace current(const allocator_type& alloc = allocator_type()) noexcept;
    static basic_stacktrace current(size_type skip, const allocator_type& alloc = allocator_type()) noexcept;
    static basic_stacktrace current(size_type skip, size_type max_depth, const allocator_type& alloc = allocator_type()) noexcept;

    basic_stacktrace() noexcept(is_nothrow_default_constructible_v<allocator_type>);
```

```
    explicit basic_stacktrace(const allocator_type& alloc) noexcept;

    basic_stacktrace(const basic_stacktrace& other);
    basic_stacktrace(basic_stacktrace&& other) noexcept;
    basic_stacktrace(const basic_stacktrace& other, const allocator_type& alloc);
    basic_stacktrace(basic_stacktrace&& other, const allocator_type& alloc);
    basic_stacktrace& operator=(const basic_stacktrace& other);
    basic_stacktrace& operator=(basic_stacktrace&& other)
        noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
        allocator_traits<Allocator>::is_always_equal::value);

    ~basic_stacktrace();

    // 20.?.5.3, observers
    allocator_type get_allocator() const noexcept;

    const_iterator begin() const noexcept;
    const_iterator end() const noexcept;
    const_reverse_iterator rbegin() const noexcept;
    const_reverse_iterator rend() const noexcept;

    const_iterator cbegin() const noexcept;
    const_iterator cend() const noexcept;
    const_reverse_iterator crbegin() const noexcept;
    const_reverse_iterator crend() const noexcept;

    [[nodiscard]] bool empty() const noexcept;
    size_type size() const noexcept;
    size_type max_size() const noexcept;

    const_reference operator[](size_type ) const;
    const_reference at(size_type ) const;

    // 20.?.5.4, comparisons
    template <class Allocator2>
    friend bool operator==(const basic_stacktrace& x, const basic_stacktrace< Allocator2 >& y) noexcept;
    template <class Allocator2>
    friend strong_ordering operator<=>(const basic_stacktrace& x, const basic_stacktrace< Allocator2 >& y) noexcept;

    // 20.?.5.5, modifiers
    void swap(basic_stacktrace& other)
        noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
        allocator_traits<Allocator>::is_always_equal::value);

  private:
    vector<value_type, allocator_type> frames_; // exposition only
  };

}
```

The class template `basic_stacktrace` satisfies the requirements of an allocator-aware container, of a sequence container and reversible container (21.2.1, 21.2.3) except that

- only move, assignment, swap and operations defined for const-qualified sequence containers are supported
- and that the semantics of comparison functions are different from those required for a container.

20.?.5.2 Creation and assignment [stacktrace.basic.cons]

```
static basic_stacktrace current(const allocator_type& alloc = allocator_type()) noexcept;
```

Returns: A `basic_stacktrace` object with `frames_` storing the stacktrace of the current evaluation in the current thread of execution, or an empty `basic_stacktrace` object if `frames_` initialization failed. `alloc` is passed to the constructor of the `frames_` object.

[Note: If the stacktrace was successfully obtained, then `frames_.front()` is the `stacktrace_entry` representing approximately the current evaluation, and `frames_.back()` is the `stacktrace_entry` representing approximately the initial function of the current thread of execution. - *end note]*

```
static basic_stacktrace current(size_type skip, const allocator_type& alloc = allocator_type()) noexcept;
```

Let `t` be a stacktrace as-if obtained via `basic_stacktrace::current(alloc)`. Let `n` be `t.size()`.

Returns: `basic_stacktrace` object with direct-non-list-initialized `frames_` from arguments `t.begin() + min(n, skip)`, `t.end()` and `alloc`, or an empty `basic_stacktrace` object if `frames_` initialization failed.

```
static basic_stacktrace current(size_type skip, size_type max_depth, const allocator_type& alloc = allocator_type()) noexcept;
```

Let `t` be a stacktrace as-if obtained via `basic_stacktrace::current(alloc)`. Let `n` be `t.size()`.

Preconditions: `skip <= skip + max_depth`.

Returns: `basic_stacktrace` object with direct-non-list-initialized `frames_` from arguments `t.begin() + min(n, skip)`, `t.begin() + min(n, skip + max_depth)` and `alloc`, or an empty `basic_stacktrace` object if `frames_` initialization failed.

```
basic_stacktrace() noexcept(is_nothrow_default_constructible_v<allocator_type>);
```

Postconditions: `empty()` is `true`.

```
explicit basic_stacktrace(const allocator_type& alloc) noexcept;
```

Effects: `alloc` is passed to the `frames_` constructor.

Postconditions: `empty()` is `true`.

```
basic_stacktrace(const basic_stacktrace& other);
basic_stacktrace(const basic_stacktrace& other, const allocator_type& alloc);
basic_stacktrace(basic_stacktrace&& other, const allocator_type& alloc);
basic_stacktrace& operator=(const basic_stacktrace& other);
basic_stacktrace& operator=(basic_stacktrace&& other)
        noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
        allocator_traits<Allocator>::is_always_equal::value);
```

Remarks: Implementations may strengthen the exception specification for those functions [res.on.exception.handling] by ensuring that `empty()` is `true` on failed allocation.

20.?.5.3 Observers [stacktrace.basic.obs]

```
using const_iterator = implementation-defined;
```

The type models random_access_iterator [iterator.concept.random.access] and meets the *Cpp17RandomAccessIterator* requirements [random.access.iterators].

```
allocator_type get_allocator() const noexcept;
```

Returns: `frames_.get_allocator()`.

```
const_iterator begin() const noexcept;
const_iterator cbegin() const noexcept;
```

Returns: An iterator referring to the first element in `frames_`. If `empty()` is `true`, then it returns the same value as `end()`.

```
const_iterator end() const noexcept;
const_iterator cend() const noexcept;
```

Returns: The end iterator.

```
const_reverse_iterator rbegin() const noexcept;
const_reverse_iterator crbegin() const noexcept;
```

Returns: `reverse_iterator(cend())`.

```
const_reverse_iterator rend() const noexcept;
const_reverse_iterator crend() const noexcept;
```

Returns: `reverse_iterator(cbegin())`.

```
[[nodiscard]] bool empty() const noexcept;
```

Returns: `frames_.empty()`.

```
size_type size() const noexcept;
```

Returns: `frames_.size()`.

```
size_type max_size() const noexcept;
```

Returns: `frames_.max_size()`.

```
const_reference operator[](size_type frame_no) const;
```

Preconditions: `frame_no < size()`.

Returns: `frames_[frame_no]`.

Throws: Nothing.

```
const_reference at(size_type frame_no) const;
```

Returns: `frames_[frame_no]`.

Throws: `out_of_range` if `frame_no >= size()`.

20.?.5.4 Comparisons [stacktrace.basic.comp]

```
template <class Allocator2>
friend bool operator==(const basic_stacktrace& x, const basic_stacktrace< Allocator2 >& y) noexcept;
```

Returns: `equal(x.begin(), x.end(), y.begin(), y.end())`

```
template <class Allocator2>
friend strong_ordering operator<=>(const basic_stacktrace& x, const basic_stacktrace< Allocator2 >& y) noexcept;
```

Returns: `x.size() <=> y.size()` if `x.size() != y.size()`. `lexicographical_compare_3way(x.begin(), x.end(), y.begin(), y.end())` otherwise.

20.?.5.5 Modifiers [stacktrace.basic.mod]

```
void swap(basic_stacktrace& other)
        noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
        allocator_traits<Allocator>::is_always_equal::value);
```

Effects: Exchanges the contents of `*this` and `other`.

20.?.6 Non-member functions [stacktrace.nonmembers]

```
template<class Allocator>
void swap(basic_stacktrace<Allocator>& a, basic_stacktrace<Allocator>& b)
        noexcept(noexcept(a.swap(b)));
```

Effects: Equivalent to `a.swap(b)`.

```
string to_string(const stacktrace_entry& f);
```

Returns: A string with a description of `f`.

[Note: The description should provide information about contained evaluation, including information from `source_file()` and `source_line()`. - *end note]*

```
template<class Allocator>
string to_string(const basic_stacktrace<Allocator>& st);
```

Returns: A string with a description of `st`.

[Note: The number of lines is not guaranteed to be equal to `st.size()`. - *end note]*

```
template<class charT, class traits>
basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const stacktrace_entry& f);
```

Effects: Equivalent to: `return os << to_string(f)`.

```
template<class charT, class traits, class Allocator>
basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const basic_stacktrace<Allocator>& st);
```

Effects: Equivalent to: `return os << to_string(st);`

20.?.7 Hash support [stacktrace.hash]

```
template<> struct hash<stacktrace_entry>;
template<class Allocator> struct hash<basic_stacktrace<Allocator>>;
```

The specializations are enabled (23.14.15).

Header <version> synopsis [version.syn]

[Note for editor: Add a macro [version.syn] ***- end note]***

```
#define __cpp_lib_stacktrace DATE_OF_ADOPTION // also in <stacktrace>
```

VI. Acknowledgments

Many thanks to Jens Maurer, JF Bastien, Marshall Clow and Tim Song for pointing out many issues in the early wordings.

Special thanks to Jens Maurer for doing the core wordings.

Many many thanks to all the people who participated in the LWG meeting on 20th of August and reviewed early version of the wording.